

Backpropagation: From Basics to Multiclass and Minibatches

1 Introduction

Backpropagation is the cornerstone algorithm for training neural networks, enabling efficient computation of gradients for weight updates. It leverages the chain rule of differentiation to propagate error signals from the output layer to the input layer, adjusting weights to minimize a loss function.

This document covers:

- The mathematical foundation of backpropagation.
- A step-by-step binary classification example with two layers.
- The generalization to multiclass problems.
- Efficient computation in practice using minibatches.

2 Network Architecture and Backpropagation Example

2.1 Network Architecture

The neural network we consider has the following structure:

- **Input Layer** (x_1, x_2): Two input features, x_1 and x_2 .
- **Hidden Layer**:
 - Two nodes:
 1. Node 1 has ReLU activation:

$$a_1 = \text{ReLU}(z_1) = \max(0, z_1).$$

2. Node 2 has tanh activation:

$$a_2 = \tanh(z_2) = \frac{\exp(z_2) - \exp(-z_2)}{\exp(z_2) + \exp(-z_2)}.$$

- Pre-activations are:

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1,$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2.$$

- **Output Layer:**

- One node with sigmoid activation:

$$\hat{y} = \sigma(z_3) = \frac{1}{1 + \exp(-z_3)}.$$

- Pre-activation:

$$z_3 = w_{31}a_1 + w_{32}a_2 + b_3.$$

2.2 Loss Function

The loss function used is the binary cross-entropy loss:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})].$$

2.3 Forward Pass

The forward pass computations are as follows:

1. Compute z_1, z_2 :

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1, \quad z_2 = w_{21}x_1 + w_{22}x_2 + b_2.$$

2. Compute a_1, a_2 :

$$a_1 = \text{ReLU}(z_1) = \max(0, z_1), \quad a_2 = \tanh(z_2) = \frac{\exp(z_2) - \exp(-z_2)}{\exp(z_2) + \exp(-z_2)}.$$

3. Compute z_3 :

$$z_3 = w_{31}a_1 + w_{32}a_2 + b_3.$$

4. Compute $\hat{y}$:

$$\hat{y} = \sigma(z_3) = \frac{1}{1 + \exp(-z_3)}.$$

5. Compute the loss:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})].$$

2.4 Backpropagation

The backward pass involves the following steps:

1. **Gradient of Loss w.r.t. $\hat{y}$:**

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}.$$

2. **Gradient of Loss w.r.t. z_3 :** Using the sigmoid derivative:

$$\frac{\partial \hat{y}}{\partial z_3} = \hat{y}(1 - \hat{y}),$$

we have:

$$\frac{\partial \mathcal{L}}{\partial z_3} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3} = \left(-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \right) \cdot \hat{y}(1 - \hat{y}).$$

3. **Gradient of Loss w.r.t.** w_{31}, w_{32}, b_3 :

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_{31}} &= \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{31}} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot a_1, \\ \frac{\partial \mathcal{L}}{\partial w_{32}} &= \frac{\partial \mathcal{L}}{\partial z_3} \cdot a_2, \\ \frac{\partial \mathcal{L}}{\partial b_3} &= \frac{\partial \mathcal{L}}{\partial z_3}.\end{aligned}$$

4. **Gradient of Loss w.r.t.** a_1, a_2 :

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot w_{31}, \quad \frac{\partial \mathcal{L}}{\partial a_2} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot w_{32}.$$

5. **Gradient of Loss w.r.t.** z_1, z_2 : For z_1 (ReLU derivative):

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1}, \quad \frac{\partial a_1}{\partial z_1} = I(z_1 > 0).$$

For z_2 (tanh derivative):

$$\frac{\partial \mathcal{L}}{\partial z_2} = \frac{\partial \mathcal{L}}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2}, \quad \frac{\partial a_2}{\partial z_2} = 1 - \tanh^2(z_2).$$

6. **Gradient of Loss w.r.t.** $w_{11}, w_{12}, b_1, w_{21}, w_{22}, b_2$: For the weights of the first layer:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_{11}} &= \frac{\partial \mathcal{L}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot x_1, \\ \frac{\partial \mathcal{L}}{\partial w_{12}} &= \frac{\partial \mathcal{L}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{12}} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot x_2, \\ \frac{\partial \mathcal{L}}{\partial b_1} &= \frac{\partial \mathcal{L}}{\partial z_1}.\end{aligned}$$

Similarly for w_{21}, w_{22}, b_2 , using $\frac{\partial \mathcal{L}}{\partial z_2}$.

2.5 Backward Pass

The gradient descent update for weights and biases is:

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}, \quad \mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}},$$

where η is the learning rate.

The gradients are computed as follows:

1. Compute $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} \odot \sigma'(\mathbf{z}^{[L]})$, where $\odot$ denotes element-wise multiplication.
2. For $l = L - 1, L - 2, \dots, 1$:

$$\delta^{[l]} = (\mathbf{W}^{[l+1]T} \delta^{[l+1]}) \odot \sigma'(\mathbf{z}^{[l]}).$$

3. Compute gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} \mathbf{a}^{[l-1]T}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \sum_i \delta_i^{[l]}.$$

3 Binary Classification Example

Consider a two-layer neural network:

- Input layer: $\mathbf{x} = [x_1, x_2]^T$.
- Hidden layer: Two nodes with ReLU and tanh activations.
- Output layer: One node with a sigmoid activation for binary classification.

3.1 Forward Pass

The computations are:

$$\begin{aligned} \mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}, & \mathbf{a}^{[1]} &= [\text{ReLU}(z_1^{[1]}), \tanh(z_2^{[1]})]^T, \\ z^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]}, & a^{[2]} &= \sigma(z^{[2]}). \end{aligned}$$

3.2 Backward Pass

1. Compute the loss derivative:

$$\frac{\partial \mathcal{L}}{\partial a^{[2]}} = a^{[2]} - y.$$

2. Propagate to the output layer:

$$\delta^{[2]} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \cdot \sigma'(z^{[2]}).$$

3. Compute gradients for the output layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[2]}} = \delta^{[2]}\mathbf{a}^{[1]T}, \quad \frac{\partial \mathcal{L}}{\partial b^{[2]}} = \delta^{[2]}.$$

4. Propagate to the hidden layer:

$$\delta^{[1]} = (\mathbf{W}^{[2]T}\delta^{[2]}) \odot [\text{ReLU}'(z_1^{[1]}), \tanh'(z_2^{[1]})]^T.$$

5. Compute gradients for the hidden layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1]}} = \delta^{[1]}\mathbf{x}^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1]}} = \delta^{[1]}.$$

4 Generalization to Multiclass Classification

For multiclass problems with K classes:

- The output layer uses a softmax activation:

$$a_k^{[L]} = \frac{e^{z_k^{[L]}}}{\sum_{j=1}^K e^{z_j^{[L]}}}, \quad k = 1, 2, \dots, K.$$

- The loss function is the cross-entropy:

$$\mathcal{L} = - \sum_{k=1}^K y_k \log a_k^{[L]}.$$

- Backpropagation adjusts gradients accordingly, with:

$$\delta_k^{[L]} = a_k^{[L]} - y_k.$$

5 Minibatch Stochastic Gradient Descent

In practice, backpropagation is performed over minibatches of size m for computational efficiency. For each minibatch:

1. Compute forward and backward passes for all m samples.
2. Average gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{1}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}_i}{\partial \mathbf{W}^{[l]}}.$$

3. Update parameters:

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}, \quad \mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}}.$$

6 The Mathematics of Backpropagation

Given a neural network with L layers, let:

- $\mathbf{x}$ be the input vector.
- $\mathbf{W}^{[l]}$ and $\mathbf{b}^{[l]}$ be the weights and biases of layer l .
- $\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$ be the pre-activation of layer l .
- $\mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]})$ be the post-activation of layer l , where σ is the activation function.

The goal is to minimize a loss function $\mathcal{L}$, which depends on the network's output $\mathbf{a}^{[L]}$ and the true labels $\mathbf{y}$.

7 Conclusion

Backpropagation is a powerful algorithm that scales efficiently with minibatches and supports both binary and multiclass tasks. This document detailed its implementation for binary classification, generalization to multiclass, and practical optimization via minibatches.